

Grundlagen digitaler Systeme WS 25/26

Hausaufgabe 4

Abgabe: 08.01.2026, 23:59

Testate: 12.01.2026–23.01.2026

Bitte geben Sie in den folgenden Aufgaben ihre Lösungsschritte stets explizit an, falls nicht anders angegeben. Nicht kommentierte Lösungen werden **nicht** gewertet.

Aufgabe 1 RISC-V Konventionen

1. Nennen Sie *mögliche* Konventionen zur Registersicherung, zu Alignment, zum Stack und zur Parameterübergabe.
2. Recherchieren Sie die *konkret* bei RISC-V verwendeten Konventionen. Setzen Sie diese bei den folgenden Aufgaben um.

Hinweis: Gehen Sie in den folgenden Aufgaben von Natural Alignment des Stacks aus.

Aufgabe 2 Unterprogramme

Geben Sie Ihren Assembler-Code der folgenden Unteraufgaben jeweils in einer separaten Assembler-Datei ab.

1. Wie könnte die Implementierung der in der Vorlesung erwähnten Semantiken für Parameter (call-by-value und call-by-reference) in Risc-V aussehen?
2. Division und Rest natürlicher Zahlen
 - a) Implementieren Sie zwei Unterprogramme $\text{mod}(a, b)$ und $\text{div}(a, b)$ in Ripes.
 - Sie dürfen die Instruktionen `div`, `rem` und `mul` nicht verwenden.
 - Sie dürfen davon ausgehen, dass die Unterprogramme nur natürliche Zahlen als Parameter bekommen.
 - b) Verändern sie die Unterprogramme nun so, dass die Parameter über call-by-reference übergeben werden.
 - c) Bonusaufgabe: Verändern sie die Unterprogramme so, dass sie auch mit negativen Zahlen funktionieren.

3. Implementieren Sie nun den Sortieralgorithmus Quicksort.

Nutzen Sie dazu quicksort.s als Grundlage. Jedes Wort ist ein Eintrag im Array, wobei l_0 die niedrigste Adresse und h_i die höchste Adresse ist.

- *Sie dürfen annehmen, dass Ihr Algorithmus nur Integer (Worte) sortieren muss.*
- *Implementieren Sie den Algorithmus mithilfe der Unterprogramme quicksort, partition und swap.*
- *Falls Sie Inspiration benötigen, orientieren Sie sich an diesem Pseudocode: [Quicksort - Hoare partition scheme](#)*

4. Stellen Sie den Stackframe Ihrer Quicksort-Implementierung zu einem Zeitpunkt während des Sortierens visuell dar.

Im Testat sollen Sie Ihrem Tutor u.a. beschreiben, wie sich der Stackframe während des Programmablaufs verändert. Bereiten Sie sich entsprechend darauf vor.

Aufgabe 3 Interrupts

Eine Interrupt Service Routine ist im Wesentlichen ein Unterprogramm mit einem ungewöhnlichen Aufruf. Für Unterprogrammaufrufe sind aus der Vorlesung bestimmte Konventionen bzw. Verantwortlichkeiten bekannt. In dieser Aufgabe widmen wir uns den Gemeinsamkeiten und Unterschieden von Unterprogrammen und Interrupts.

1. Welche Arten von Interrupts gibt es? Wodurch werden diese ausgelöst?
2. Wie bzw. wann wird auf das Eintreten eines Interrupts reagiert? Wie unterscheiden sich die Arten hier? Auf welcher Betrachtungsebene (Mikroinstruktionen, Instruktionen, Programme, ...) erfolgt dies?
3. Was ergibt sich daraus für die Verantwortlichkeiten im Falle eines Interrupts? Gehen Sie zudem auf die Sinnhaftigkeit von Rückgabewerten ein.
4. Es gibt zudem die Möglichkeit, Interrupts zu sperren oder zu priorisieren, sodass die Behandlung nicht umgehend erfolgen muss. Welchen Nutzen könnte dies haben?
5. Gehen Sie von Interruptbehandlung anhand einer Sprungtabelle aus, von welcher in die richtige Interrupt Service Routine gesprungen wird.
 - a) Angenommen, die Sprungtabelle befände sich ohne Schreibschutz im Hauptspeicher. Welches Sicherheitsrisiko ergibt sich daraus? Welche Differenzierung wird nötig?
 - b) Stellen Sie den Prozess der Ausnahmebehandlung auf eine Ihres Erachtens geeignete Weise dar; schematisch, textuell, ...
 - c) Wie endet die Behandlung einer Ausnahme? Was muss der Prozessor hier leisten?